

Methodology

Methodology I

This section describes the implementation methodology, explicitly detailing how the optimization algorithms are constructed, validated, and analyzed using the Generalized Research Template's infrastructure and tests ecosystems.

Algorithm Implementation I

Algorithm Implementation II

Gradient Descent Algorithm

The core algorithm implements the iterative procedure for unconstrained optimization. The optimizer module uses the standard-library logging logger for optional verbose diagnostics; the analysis orchestrator uses `infrastructure.core.logging.utils.get_logger`. Tests run under the hermetic boundaries defined in the test configuration.

Algorithm — Gradient Descent (implemented in the optimizer module)

Input: Initial point x_0 , step size α , tolerance ϵ , max iterations $N_{\max}$

1. Initialize $k \leftarrow 0$
2. **While** $k < N_{\max}$ **do:**
 - ▶ Compute gradient $g_k = \nabla f(x_k)$
 - ▶ **If** $\|g_k\|_2 < \epsilon$ **then return** x_k (converged)
 - ▶ Update: $x_{k+1} \leftarrow x_k - \alpha \cdot g_k$
 - ▶ $k \leftarrow k + 1$
3. **Return** x_k (max iterations reached)

Infrastructure Integration I

The methodology explicitly bridges theoretical mathematics with production-grade validation through the `infrastructure.scientific` module.

Numerical Stability Analysis

Rather than writing ad-hoc validation code, the project imports `infrastructure.scientific.stability.check_numerical_stability`. This utility subjects the objective function to a barrage of extreme inputs (NaN, Inf, $\pm 10^{10}$) to calculate a formalized stability score. If this score degrades, the analysis orchestrator execution deliberately aborts, ensuring the methodology cannot enter unrecoverable states.

Infrastructure Integration II

Performance Benchmarking

The analysis exercises `infrastructure.scientific.benchmarking.benchmark_function` as a runtime diagnostic.

Host-dependent timing and memory observations are logged but excluded from tracked evidence. The canonical report records fixed inputs and exact objective values, while the dimensional figure reports deterministic convergence iterations and the proxy $d \times \text{iterations}$; neither surface claims that one host run proves an asymptotic bound.

Convergence Analysis I

For quadratic functions $f(x) = \frac{1}{2}x^T Ax - b^T x$ where A is positive definite, the convergence factor becomes [bertsekas1999nonlinear]:

$$\rho = \frac{|\lambda_{\max} - \alpha\lambda_{\min}|}{|\lambda_{\min} + \alpha\lambda_{\max}|} \quad (1)$$

Optimal convergence occurs when $\alpha = \frac{2}{\lambda_{\min} + \lambda_{\max}}$, yielding

$$\rho = \frac{\kappa - 1}{\kappa + 1}.$$

Experimental Setup I

Experimental Setup II

Step Size Analysis

Step sizes are not chosen ad hoc in the manuscript: they are read from `experiment.step_sizes` in `manuscript/config.yaml` and passed through `run_convergence_experiment()` in `src/analysis/` (entry: `scripts/optimization_analysis.py`).

The active grid for this build is:

- ▶ $\alpha = 0.01$ (conservative)
- ▶ $\alpha = 0.1$ (conservative)
- ▶ $\alpha = 0.5$ (near-optimal)
- ▶ $\alpha = 1.0$ (near-optimal)
- ▶ $\alpha = 1.5$ (aggressive)
- ▶ $\alpha = 2.5$ (divergent (expected unstable for $H = I$))

Labels follow the same agency taxonomy used for plot colours (`_agency_category` in the analysis script): **conservative** (small α , slow but stable on $H = I$), **near-optimal** (including $\alpha = 1$ where the method reaches x^* in one step for this quadratic), **aggressive** ($1 < \alpha < 2$, still linearly contracting in $|x - x^*|$ but oscillatory in sign), and **divergent** ($|1 - \alpha| \geq 1$).

Analysis Pipeline & LaTeX Integration I

The automated analysis script leverages `infrastructure.core.progress` (`ProgressBar`, `SubStageProgress`) to orchestrate experiments, collect convergence trajectories, and generate publication-quality visualizations seamlessly.

The research template supports advanced LaTeX customization through the `preamble.md` configuration. This is ingested directly by `infrastructure.rendering.latex_utils.py` and `pdf_renderer.py`, automatically linking compiled PGF plots and BibTeX citations. This automated approach ensures an unbreakable chain of custody from raw algorithmic execution to the final rendered manuscript.